

R codes for the NutNet-Stability project

Step 3: Analysis of single stability responses

Qiang Yang

2026-06-22

Overview

This document analyses how individual components of ecological stability respond to nutrient enrichment intensity and treatment duration. It starts from the stability dataset generated in Step 2 and fits separate models for each single-nutrient addition treatment (nitrogen, phosphorus, and potassium) and each stability response.

The analyses focus on the four stability metrics used in the manuscript: functional temporal variability, extent of functional change, compositional temporal variability, and extent of compositional change. The workflow scales the stability responses, standardizes predictors, fits linear mixed-effects models with site as a random intercept, extracts regression coefficients and significance tests, and generates figures summarizing coefficient estimates and model predictions.

These analyses provide the statistical basis for evaluating whether relative nutrient enrichment intensity, treatment duration, and their interaction influence each dimension of grassland stability. The code is annotated to clarify the purpose of each processing, modelling, and plotting step while preserving the analytical workflow used in the manuscript.

Output

Running this workflow generates the following outputs:

1. A regression-coefficient figure showing the effects of relative nutrient enrichment intensity, treatment duration, and their interaction on the four stability metrics used in the manuscript (Fig. 2).
2. A model-prediction figure showing fitted relationships between enrichment intensity and stability responses at different treatment durations. We only included significant interactions between enrichment intensity and treatment duration to faithfully reproduce the figure included in the manuscript (Fig. 3).

Section 1: Set up the working environment

```
rm(list = ls())
gc()
```

```
##           used (Mb) gc trigger (Mb) max used (Mb)
## Ncells 496400 26.6   1075080 57.5   686411 36.7
## Vcells 927178  7.1    8388608 64.0  1876578 14.4
```

```

# necessary R packages
Packages <- c(
  "tidyverse", "magrittr", "grid", "gridExtra",
  "scales", "broom", "nlme", "sjPlot", "patchwork", "lme4", "lmerTest"
)
pacman::p_load(char = Packages)
# functions with conflicting status (presence in different packages used here)
select <- dplyr::select
summarise <- dplyr::summarise

# working path
main.path <- getwd()
data.path <- paste0(main.path, "/mydata")
figure.path <- paste0(main.path, "/myfigure")
set.seed(1234)

```

Section 2: Prepare the stability dataset for modelling

```

load(paste0(data.path, "/df.tidy.stabilities.Rdata"))
load(paste0(data.path, "/df.tidy.Rdata"))
df.full <- df.tidy.stabilities %>%
  left_join(df.tidy %>% select(-contains("data")))
rm(df.tidy, df.tidy.stabilities)

df.full %<>%
  select(site_code:end_year, matches("_added_proportion|change|var"))
sta.names <- colnames(df.full) %>% str_subset("change|var")

# Scale the stability components before modelling.
# Some variables contain Inf or NA values, so this helper function first
# converts infinite values to NA and then applies z-standardization.
new.scale <- function(x) {
  x[is.infinite(x)] <- NA
  y <- scale(x)
}

```

```

df.full <- df.full %>%
  mutate(
    func_var = new.scale(func_var),
    func_var_det = new.scale(func_var_det),
    comp_var = new.scale(comp_var),
    func_change = new.scale(func_change),
    comp_change = new.scale(comp_change)
  )

# Create nested datasets for the three single-nutrient addition treatments: N, P, and K.
df.meta.N.P.K <- data.frame(trt = c("N", "P", "K")) %>%
  mutate(pert.name = paste0(trt, "_added_proportion")) %>%
  expand_grid(sta.names)

df.N.P.K.nested <- df.meta.N.P.K %>%
  mutate(
    data = pmap(list(trt, pert.name, sta.names), function(x1, x2, x3) {
      output <- df.full %>%
        filter(trt == x1) %>%
        select(one_of(c("site_code", "block", "plot", "end_year", x2, x3))) %>%
        set_colnames(c("site_code", "block", "plot", "end_year", "perturbation_intensity", "stability"))
    })
  )

# Some stability values are missing because biomass or cover data are unavailable
# for particular plots and time windows. Remove these missing values before modelling.
df.N.P.K.nested <- df.N.P.K.nested %>%
  mutate(data = map(data, na.omit))
# Remove infinite values, which can arise from ratios or transformations when
# the denominator is zero or effectively unavailable.
df.N.P.K.nested <- df.N.P.K.nested %>%
  mutate(data = map(data, function(df1) df1 %>% filter(!is.infinite(stability))))

```

Section 3: Fit models with site as a random intercept

For each nutrient treatment and stability response, a linear mixed-effects model is fitted with relative enrichment intensity, treatment duration, and their interaction as fixed effects. Site is included as a random intercept to account for baseline differences among NutNet sites.

3.1 Fit the regression models

```
df.models.N.P.K <- df.N.P.K.nested %>%
  mutate(mod.out = map(data, function(dat) {
    # Log-transform relative enrichment intensity and remove incomplete records.
    dat %<>% mutate(log.pert = log(perturbation_intensity))
    dat %<>% na.omit()
    mod.dat <- dat

    # Standardize both continuous predictors to make coefficient estimates comparable.
    mod.dat %<>% mutate(across(c(end_year, log.pert), scale))

    # Model each stability response as a function of enrichment intensity,
    # treatment duration and their interaction, with site as a random intercept.
    formula_ <- stability ~ log.pert + end_year + log.pert:end_year + (1 | site_code)
    mod <- lmer(formula = formula_, data = mod.dat)
    return(mod)
  })))
```

There is no convergence problem for this set of models.

3.2 Organize model coefficient estimates and P values

```
df.models.N.P.K.coefficients <- df.models.N.P.K %>%
  mutate(mod.coefficient = map(mod.out, function(mod) {
    # Extract fixed-effect coefficient estimates from the model summary.
    output <- summary(mod)$coefficients
    coeff <- output %>% as.data.frame()
    coeff %<>% mutate(predictor = rownames(output))
    coeff %<>% select(predictor, Estimate)
    # Extract confidence intervals for fixed-effect estimates using lme4::confint.merMod().
    cfs <- confint.merMod(mod)
    cfs %<>% as.data.frame()
    cfs %<>% mutate(predictor = rownames(cfs))
    cfs %<>% rename(lcf = `2.5 %`, ucf = `97.5 %`)
    cfs %<>% filter(!str_detect(predictor, ".sig"))
  })))
```

```

# Extract F-tests and P values using lmerTest.
df.p.value <- anova(mod) %>% as.data.frame()
df.p.value %<>% select(NumDF:`Pr(>F)`)
df.p.value %<>% mutate(predictor = rownames(df.p.value))

# Classify statistically significant positive and negative effects for plotting.
coeff <- coeff %>%
  left_join(cfs) %>%
  left_join(df.p.value)
coeff %<>% filter(predictor != "(Intercept)")
coeff %<>%
  mutate(sig.sign = map2_chr(Estimate, `Pr(>F)`, function(x, p) {
    if (x < 0 & p < 0.05) {
      out <- "negative"
    } else if (x > 0 & p < 0.05) {
      out <- "positive"
    } else {
      out <- "non-significant"
    }
    return(out)
  }))
return(coeff)
}))

df.models.N.P.K.coefficients %<>%
  mutate(model.ID = 1:nrow(df.models.N.P.K.coefficients)) %>%
  select(-pert.name, -data, -mod.out) %>%
  select(model.ID, everything()) %>%
  unnest(cols = mod.coefficient)

```

3.3 Visualize model coefficient estimates

```

df.coefficients <- df.models.N.P.K.coefficients %>%
  mutate(predictor = str_replace_all(predictor, "end_year", "years")) %>%
  mutate(predictor = str_replace_all(predictor, "log.pert", "perturbation"))

```

```

# Relabel the stability metrics for the regression-coefficient figure.
# The figure focuses on the four stability metrics used in the manuscript,
# and the labels are broken into three lines to improve readability in the facet strips.
stability_order <- c("func_var_det", "func_change", "comp_var", "comp_change")
stability_labs <- c(
  "func_var_det" = "Functional\ntemporal\nvariability",
  "func_change" = "Extent of\nfunctional\nchange",
  "comp_var" = "Compositional\ntemporal\nvariability",
  "comp_change" = "Extent of\ncompositional\nchange"
)

df.coefficients %<>%
  filter(sta.names %in% stability_order) %>%
  mutate(sta.names = factor(sta.names, levels = stability_order, labels = stability_labs)) %>%
  mutate(trt = as.factor(trt)) %>%
  mutate(trt = fct_relevel(trt, c("N", "P", "K"))) %>%
  mutate(predictor = as.factor(predictor)) %>%
  mutate(predictor = fct_relevel(predictor, c("years", "perturbation", "perturbation:years")))

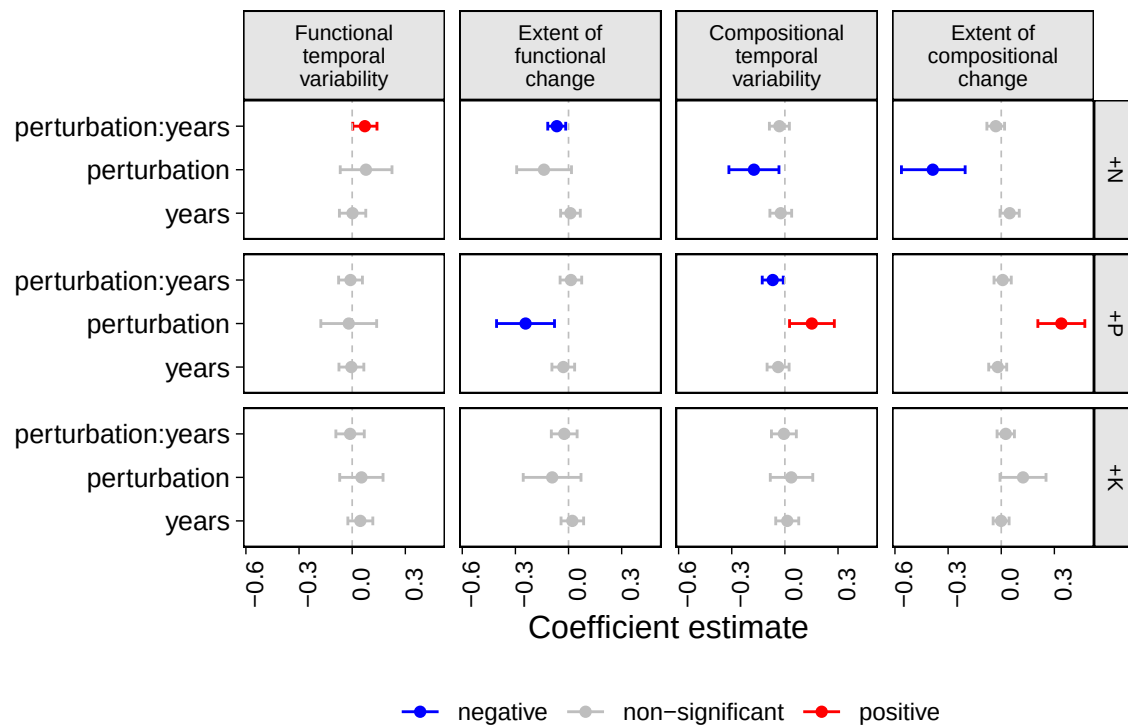
p1 <- df.coefficients %>%
  # filter(str_detect(sta.names, "functional")) %>%
  ggplot(aes(x = predictor, y = Estimate, color = sig.sign)) +
  geom_point() +
  geom_errorbar(aes(ymin = lcf, ymax = ucf), width = .2) +
  facet_grid(trt ~ sta.names) +
  coord_flip() +
  geom_hline(yintercept = 0, col = "grey", linetype = "dashed", size = 0.3) +
  scale_color_manual(values = c("blue", "grey", "red")) +
  theme_linedraw() +
  theme(
    strip.background = element_rect(fill = "gray90"),
    strip.text = element_text(color = "black", size = 8),
    panel.grid = element_blank(),
    axis.text.x = element_text(size = 9, color = "black", angle = 90),
    axis.text.y = element_text(size = 10, color = "black"),
    axis.title.x = element_text(size = 12),
    axis.title.y = element_blank(),
    plot.title = element_text(face = "bold", color = "blue"),
  )

```

```
    legend.position = "bottom",
    legend.title = element_blank()
  ) +
  xlab("Predictors") +
  ylab("Coefficient estimate")

trt.labs <- c("+N", "+P", "+K")
names(trt.labs) <- c("N", "P", "K")

# Add treatment labels to the rows of the coefficient figure.
(p1 <- p1 + facet_grid(
  trt ~ sta.names,
  labeller = labeller(trt = trt.labs)
))
```

3.4 Visualize model predictions

Model predictions are visualized using `sjPlot::plot_model()`. Predictions are shown across the range of enrichment intensity while treatment duration is represented at the mean and at one standard deviation below and above the mean, following common practice for visualizing interactions between continuous predictors.

```
p.list1 <- list() # with confidence interval
df.models.N.P.K2 <- df.models.N.P.K

# Calculate empirical ranges of the stability responses for reference when plotting predictions.
df.range <- df.full %>%
  select(contains("var"), contains("change")) %>%
  gather(key = "sta.names") %>%
```

```

na.omit()
df.range <- df.range %>% filter(!is.infinite(value))
df.range <- df.range %>%
  group_by(sta.names) %>%
  summarise(
    y.min = quantile(value, 0.05),
    y.max = quantile(value, 0.95)
  ) %>%
  ungroup()

df.models.N.P.K2 <- df.models.N.P.K2 %>% left_join(df.range)

# df.models.N.P.K2 %<% filter(str_detect(sta.names, "Functional"))
for (i in 1:15) {
  mod <- df.models.N.P.K2$mod.out[[i]]
  p <- plot_model(mod, type = "pred", terms = c("log.pert [-2,2]", "end_year")) +
    theme_bw() +
    scale_color_manual(values = c("#56B4E9", "black", "#E69F00")) +
    scale_fill_manual(values = c("#56B4E9", "black", "#E69F00")) +
    # ylim(c(df.models.N.P.K2$y.min[[i]], df.models.N.P.K2$y.max[[i]])) +
    xlab(paste0("Perturbation (+", df.models.N.P.K2$trt[[i]], ")")) +
    ylab(df.models.N.P.K2$sta.names[[i]]) +
    ggtitle("") +
    theme(
      axis.text = element_text(color = "black"),
      panel.grid = element_blank()
    ) +
    ylim(-1.3, 1.3) +
    labs(color = "Year")
  p.list1[[i]] <- p
}

```

Generate a prediction figure for cases in which the interaction between enrichment intensity and treatment duration is statistically significant. The other cases are not included in the figure to faithfully reproduce the figure included in the manuscript, but can be displayed by calling

```

Fig <- wrap_plots(p.list1[[2]] + ylab("Functional \ntemporal variability"),
  p.list1[[4]] + ylab("Extent of \nfunctional change"),
  p.list1[[8]] + ylab("Compositional\n temporal variability"),

```

```

nrow = 1, ncol = 3)
(Fig <- Fig +
  plot_annotation(tag_levels = "a") +
  plot_layout(guides = "collect") & theme(legend.position = "bottom", legend.title = element_blank()))

```

